Challenge :	The Auror’s False Name
Category :	Web Exploitation
Difficulty :	Easy
Tools Used :	Chrome browser, Burp Suite

Summary

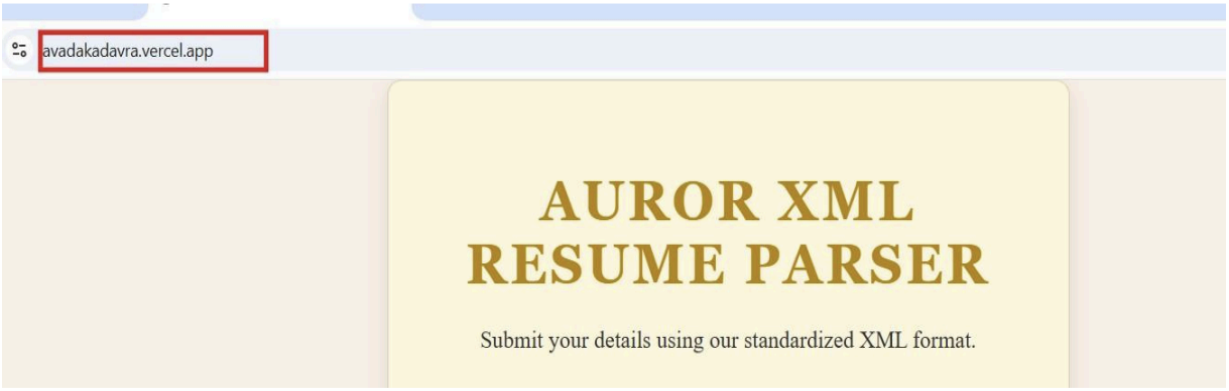
This challenge demonstrates an In-Band XML External Entity (XXE) injection vulnerability. The website takes resume information in an XML format and overly trusts the input, allowing users to access the backend by creating customized functions. By crafting a custom Document Type Definition (DOCTYPE), we create a shortcut to define an external entity that reads local files. Since the application reflects the applicant's name back in the UI, placing this entity within the <name> tag allows us to extract the hidden flag stored in /tmp/flag.txt.

Problem Statement

The Department of Magical Law Enforcement has set up a new portal to accept Auror resumes. To ensure standardized data, the portal requires applicants to submit their details in a strict XML format. However, the parser is overly trusting of the magical input it receives.

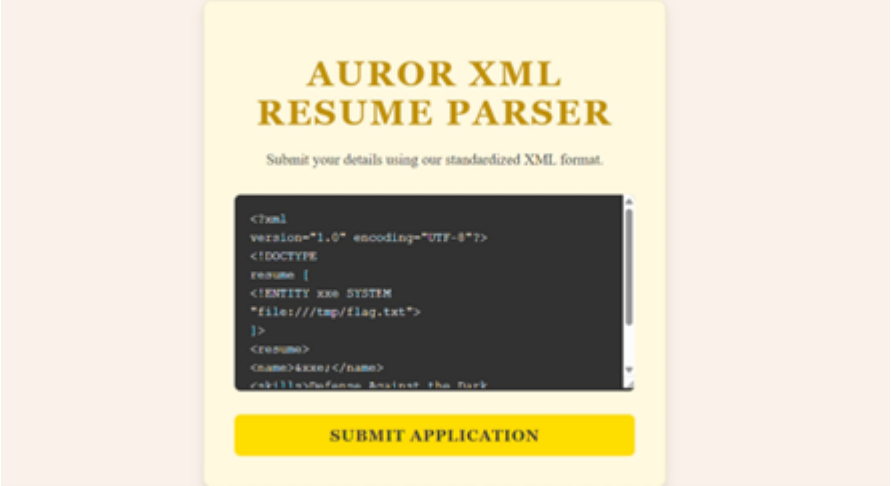
Steps to Solve

1. Open the given challenge URL in your browser.



2. Observe the "Auror XML Resume Parser" interface, which accepts user details via a standardized XML format.
3. Test the input to see how the server responds. Submitting a standard XML payload reveals that whatever text is placed inside the <name> tags is directly reflected to the user in the "System Response" message on the frontend.
4. Exploit the XML parser's vulnerability by injecting a malicious <!DOCTYPE> definition at the top of the XML code.
5. Inside the DOCTYPE, define an entity/password named xxe. Use the SYSTEM keyword to point this entity to the target local file: file:///tmp/flag.txt.
6. Place the defined &xxe; entity inside the <name> tags so the server is forced to read it and echo it back in the UI.

7. Write and submit the following malicious XML payload:
   version="1.0" encoding="UTF-8"?>
   <!DOCTYPE resume [
   <!ENTITY xxe SYSTEM "file:///tmp/flag.txt">
   ]>
   <resume>
   <name>&xxe;</name>
   <skills>Defense Against the Dark Arts</skills>
   </resume>



8. When the server parses this XML, it replaces the &xxe; reference with the actual contents of the local file.
9. Wherever the entity is present in the name tag, the information from the /tmp/flag.txt file is displayed on the frontend feedback area.
10. The response box reveals the successful extraction:
    System Response: Application received for: Cruxhunt{xxe_p4rs3r_m4g1c}

Final FLAG: Cruxhunt{xxe_p4rs3r_m4g1c}